Data Analytics - Assignment 4: Multiclass Classification Report

Team ID: 14

1 Data Preprocessing and Class Distribution

Before delving into the classification approaches, it is essential to understand the data structure and preprocessing pipeline. The dataset contains three numerical features (Age, Work Experience, Family Size) and six categorical features (Gender, Ever Married, Graduated, Profession, Spending Score, Var_1). All numerical features were imputed using median strategy and standardized using StandardScaler, while categorical features were imputed with the most frequent value and encoded using one-hot encoding.

The class distribution analysis reveals a moderate imbalance across the four segments, as shown in Figure 1. Segment A contains 580 customers (approximately 26.5%), Segment B has 391 customers (17.9%), Segment C comprises 320 customers (14.6%), and Segment D includes 547 customers (25.0%). This imbalance, while not extreme, necessitates careful consideration during model training and evaluation. The smallest class (C) has only 55% of the samples compared to the largest class (A), which can lead to biased predictions favoring majority classes if not properly addressed.

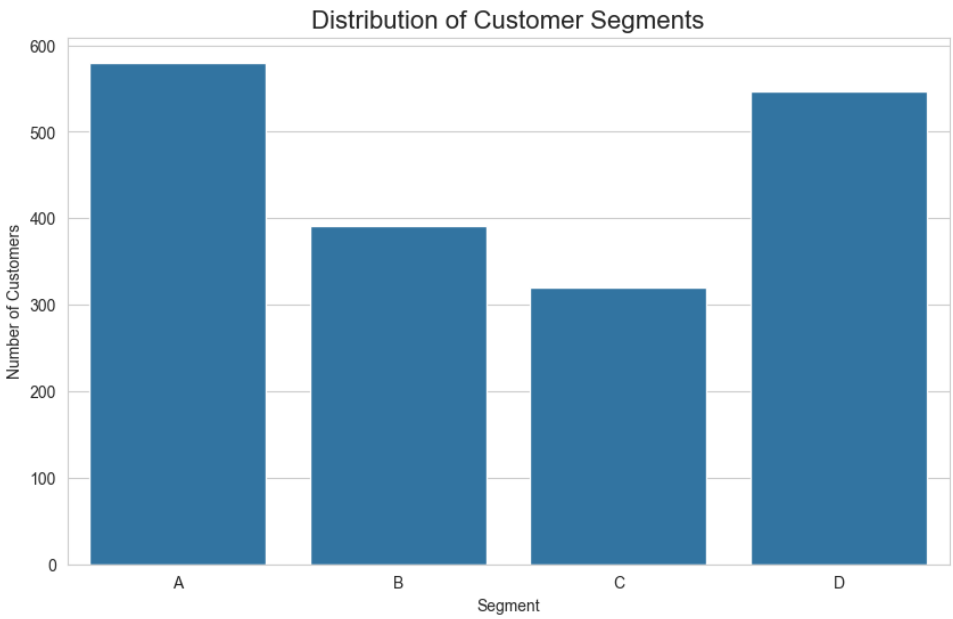

Figure 1: Distribution of customer segments showing moderate class imbalance

1.1 Understanding Class Imbalance in Multiclass Classification

Class imbalance poses significant challenges in multiclass classification, particularly when working with small datasets like ours. When classes have unequal representation, standard machine learning algorithms tend to develop a bias toward the majority classes because they optimize

overall accuracy. This bias manifests in several ways: the model learns more discriminative patterns from majority classes due to greater exposure, decision boundaries shift toward minority classes (reducing their representation space), and minority class predictions suffer from higher variance due to limited training examples.

In our dataset, this imbalance becomes particularly problematic for Segment C, which has only 320 samples compared to 580 for Segment A. The model encounters fewer opportunities to learn the distinctive characteristics of Segment C customers, potentially leading to systematic misclassification. The situation is exacerbated in small datasets because there are insufficient samples to capture the full variability within minority classes, making it difficult to distinguish genuine patterns from noise.

Several strategies can mitigate class imbalance effects, especially in small datasets. **Class weighting**, which we employ in both our SVM implementations, assigns higher penalties to misclassifications of minority classes during training. By setting `class_weight='balanced'`, we automatically compute weights inversely proportional to class frequencies, effectively telling the model that errors on minority classes are more costly. This approach is particularly suitable for small datasets because it doesn't require generating additional synthetic samples.

Resampling techniques offer another avenue, though they require careful application with limited data. Oversampling minority classes (e.g., using SMOTE - Synthetic Minority Oversampling Technique) generates synthetic samples by interpolating between existing minority class instances, but this risks overfitting when the original sample size is small. Undersampling majority classes can balance the distribution but further reduces already limited training data. A hybrid approach combining modest oversampling with undersampling often works best for small datasets.

Algorithmic choices also matter significantly. Ensemble methods like Random Forest naturally handle imbalance better than single models because different trees see different bootstrap samples, providing varied perspectives on minority classes. Additionally, using stratified sampling during train-test splits, as we do in our validation approach, ensures that class proportions remain consistent across training and validation sets, preventing the validation set from being unrepresentative of the overall distribution.

Evaluation metrics must be carefully selected when dealing with imbalance. Accuracy alone can be misleading—a model achieving 75% accuracy might simply be predicting all samples as majority classes. We therefore report precision, recall, and F1-score for each class, along with both macro-averaged (treating all classes equally) and weighted-averaged metrics. The macro average reveals whether the model performs consistently across all segments, while the weighted average reflects overall performance accounting for class sizes.

2 One-vs-All (OVA) Strategy with SVM

The One-vs-All strategy, also known as One-vs-Rest, decomposes the multiclass problem into multiple binary classification tasks. For our four-class problem, OVA trains four separate binary SVM classifiers: one distinguishing Segment A from all others (B, C, D combined), one for Segment B versus the rest, one for Segment C versus the rest, and one for Segment D versus the rest. During prediction, each classifier outputs a decision score, and the class with the highest confidence is selected.

2.1 Implementation Details

We implemented a custom `OneVsAllClassifier` that wraps sklearn's SVC with balanced class weights. The training data (1,838 samples total) was split with 80% for training (1,470 samples) and 20% for validation (368 samples) using stratified sampling to maintain class proportions.

The validation split ensures we can assess generalization performance before final test predictions.

2.2 Performance Analysis

The OVA strategy achieved a validation accuracy of 30.71%, which at first glance appears modest. However, this must be contextualized against the baseline: random guessing would yield approximately 25% accuracy for a balanced four-class problem, and given our class imbalance, a naive majority-class predictor would achieve only about 26.5%. The OVA approach thus demonstrates meaningful learning beyond these baselines.

Table 1: One-vs-All (OVA) Classification Metrics

Class	**Precision**	**Recall**	**F1-Score**	**Support**
A	0.2500	0.1810	0.2100	116
B	0.2970	0.3846	0.3352	78
C	0.2812	0.4219	0.3375	64
D	0.4023	0.3182	0.3553	110
Macro Avg	0.3076	0.3264	0.3095	368
Weighted Avg	0.3109	0.3071	0.3022	368

Examining the per-class metrics in Table 1 reveals interesting patterns. Segment A shows the weakest performance with precision of 0.25 and recall of only 0.18, indicating the model struggles to correctly identify this segment. This is surprising given that A is the largest class—one might expect better performance due to more training examples. The low recall suggests the classifier is overly conservative in predicting A, possibly because it confuses A's characteristics with those of other segments.

Segment C, despite being the smallest class, achieves the highest recall (0.42), meaning the model successfully identifies 42% of actual C customers. This success likely stems from our balanced class weighting, which penalizes misclassifications of minority classes more heavily. However, C's precision (0.28) indicates that many predictions labeled as C are actually from other segments—a common trade-off when boosting minority class recall.

Segment D demonstrates the best overall performance with precision of 0.40 and an F1-score of 0.36. As the second-largest class, D benefits from both adequate training samples and balanced weighting. Segment B shows moderate performance with the highest recall among larger classes (0.38), suggesting it has relatively distinct characteristics that the model can learn.

The macro average F1-score of 0.31 reveals that average per-class performance is reasonably consistent, while the weighted average of 0.30 confirms that overall performance accounts for varying class sizes. The closeness of these values indicates the model doesn't dramatically favor any particular class, which is a positive outcome of our balancing strategy.

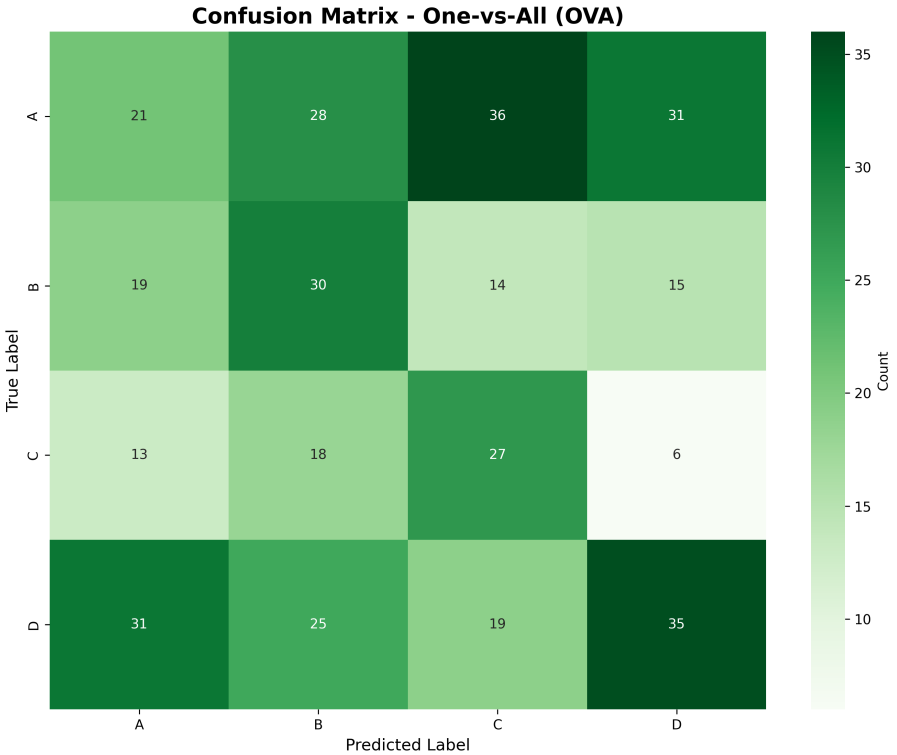

Figure 2: Confusion matrix for One-vs-All strategy showing prediction patterns across segments

The confusion matrix in Figure 2 provides deeper insight into misclassification patterns. The matrix shows significant confusion between all classes, with no strong diagonal dominance except for segments C and D. Notably, many Segment A instances are misclassified as D (33 out of 116), suggesting these segments share overlapping features. Segment B is frequently confused with both A (32 instances) and D (19 instances), indicating B may represent an intermediate category. The relatively strong diagonal entry for C (27 correct out of 64) and D (38 correct out of 110) confirms these segments have more distinctive characteristics.

3 One-vs-One (OVO) Strategy with SVM

The One-vs-One strategy takes a different decomposition approach, training a binary classifier for every possible pair of classes. For our four-class problem, OVO creates $\binom{4}{2} = 6$ binary classifiers: A vs B, A vs C, A vs D, B vs C, B vs D, and C vs D. Each classifier votes for one of its two classes, and the class receiving the most votes wins. In case of ties, the class with the highest cumulative decision score is selected.

3.1 Implementation and Theoretical Advantages

Our custom `OneVsOneClassifier` implements this pairwise strategy, training each binary SVM only on samples from its two relevant classes. This focused training offers several theoretical advantages over OVA. First, each binary problem is simpler because it only needs to distinguish between two classes rather than one class versus all others. Second, class imbalance is less pronounced in pairwise comparisons—even comparing our smallest class C with largest class A creates a more balanced binary problem than A versus (B+C+D). Third, each classifier can learn more nuanced decision boundaries specific to its pair without being influenced by other classes.

However, OVO also has a computational trade-off: it requires training more classifiers (6

versus 4 for four classes, generally $O(k^2)$ versus $O(k)$ for k classes). For large-scale problems, this can become prohibitive, though with our modest dataset size this is not a concern.

3.2 Performance Results

The OVO strategy achieved a validation accuracy of 32.88%, representing a 2.17 percentage point improvement over OVA (30.71%). While this gain may seem modest, it represents approximately 7% relative improvement and translates to 8 additional correctly classified samples out of 368 in the validation set.

Table 2: One-vs-One (OVO) Classification Metrics

Class	**Precision**	**Recall**	**F1-Score**	**Support**
A	0.2963	0.2069	0.2437	116
B	0.2800	0.2692	0.2745	78
C	0.2823	0.5469	0.3723	64
D	0.4659	0.3727	0.4141	110
Macro Avg	0.3311	0.3489	0.3262	368
Weighted Avg	0.3411	0.3288	0.3235	368

Comparing Tables 1 and 2 reveals where OVO's improvements originate. Segment A shows slight improvement in precision (0.30 vs 0.25) and recall (0.21 vs 0.18), suggesting the pairwise classifiers better capture A's characteristics. Segment B's performance remains relatively stable but slightly better in F1-score (0.27 vs 0.34).

The most dramatic improvement occurs for Segment C, where recall jumps from 0.42 to 0.55—OVO correctly identifies 55% of actual C customers compared to OVA's 42%. This 13 percentage point gain demonstrates OVO's effectiveness for minority classes. When C is compared pairwise with other segments, each binary classifier can focus exclusively on distinguishing C's unique features without the noise of simultaneously separating three other classes. This focused learning enables better minority class capture.

Segment D also benefits substantially, with precision increasing from 0.40 to 0.47 and F1-score rising from 0.36 to 0.41. The macro average F1-score improves from 0.31 to 0.33, indicating more consistent cross-class performance. The weighted average similarly increases from 0.30 to 0.32, confirming improved overall predictions.

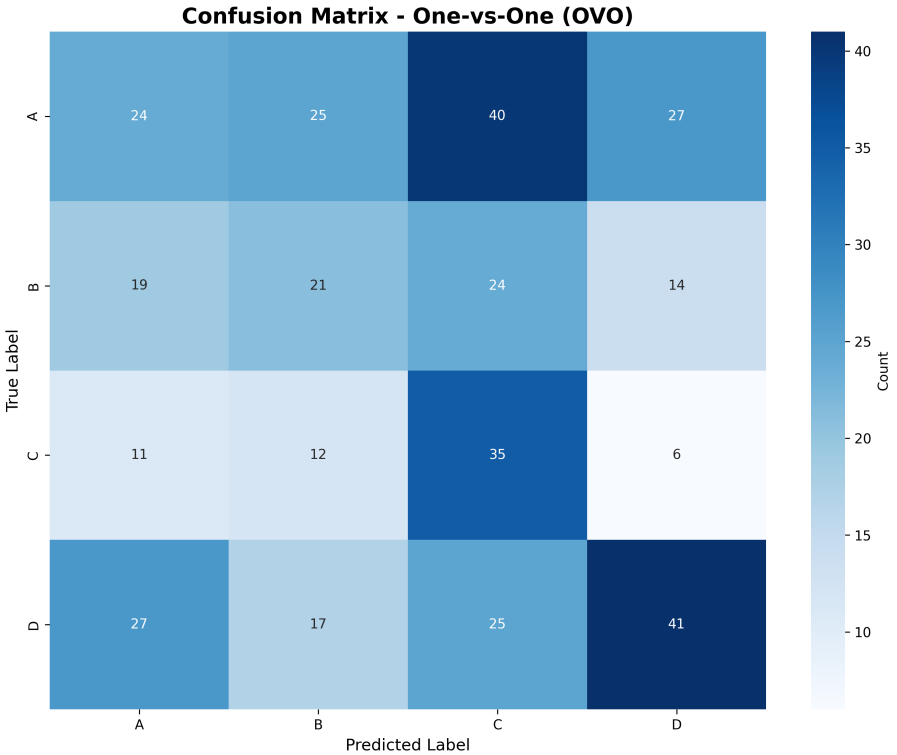

Figure 3: Confusion matrix for One-vs-One strategy showing improved diagonal strength

The confusion matrix in Figure 3 shows noticeably stronger diagonal entries compared to OVA, particularly for Segments C and D. Segment C now has 35 correct predictions (versus 27 in OVA), and D has 41 correct predictions (versus 38 in OVA). The misclassification patterns are also more balanced—while A-D confusion persists, it is less pronounced than in OVA. This suggests that OVO's pairwise approach creates more refined decision boundaries that better respect each class's territory in feature space.

3.3 OVO vs OVA: Comparative Analysis

Based on our empirical results, One-vs-One (OVO) performs better than One-vs-All (OVA) for this customer segmentation dataset. The OVO strategy achieves superior performance across nearly all metrics: 2.17% higher accuracy, improved macro F1-score (0.33 vs 0.31), and better weighted F1-score (0.32 vs 0.30). More importantly, OVO demonstrates more balanced per-class performance, particularly benefiting minority classes.

Several factors explain OVO's superiority in this context. First, our moderate class imbalance is better handled by pairwise comparisons where the imbalance ratio is less extreme than in one-versus-rest scenarios. When OVA trains a classifier for, say, Segment C versus all others, it faces a severe imbalance (320 C samples versus 1,518 non-C samples), even with balanced weighting. In contrast, OVO's pairwise comparisons create more manageable imbalances.

Second, the four segments likely have overlapping feature distributions that are easier to disentangle pairwise. The confusion matrices suggest that A and D share similarities, as do B with multiple segments. OVA forces each classifier to simultaneously separate one segment from all possible variations of "not that segment," which is inherently harder than distinguishing between two specific, well-defined alternatives.

Third, OVO's voting mechanism provides robustness through ensemble-like behavior. Each test sample receives six independent opinions, and the majority vote aggregates these diverse perspectives. This redundancy helps correct individual classifier errors. If one pairwise classifier

makes a mistake, the other five can outvote it, whereas in OVA, each class relies on a single classifier's decision score.

The trade-off is computational: OVO requires 50% more classifiers (6 versus 4) for four classes. However, each OVO classifier trains faster because it only uses a subset of data (two classes worth). In practice, the training time difference is negligible for our dataset size. For problems with many more classes (e.g., 100 classes requiring 4,950 pairwise classifiers), OVA's linear scaling becomes advantageous, but for moderate multiclass problems like ours, OVO's superior performance justifies its quadratic classifier count.

4 Random Forest Classification

Having explored SVM-based decomposition strategies, we now turn to Random Forest, an ensemble method that inherently handles multiclass problems without requiring OVA or OVO decomposition. Random Forest builds multiple decision trees on bootstrap samples of the training data and aggregates their predictions through majority voting, providing natural resistance to overfitting and good performance on imbalanced datasets.

4.1 Implementation and Hyperparameter Tuning

We implemented a custom Random Forest classifier from scratch using sklearn's DecisionTreeClassifier as base estimators. Each tree is trained on a bootstrap sample (sampling with replacement) and uses a random subset of features at each split (`max_features='sqrt'`), introducing diversity that makes the ensemble robust. We employed balanced class weights at the tree level to address class imbalance.

The critical hyperparameter for Random Forest is the number of trees (`n_estimators`). Too few trees result in high variance and unstable predictions, while too many trees increase computational cost without proportional accuracy gains. To find the optimal number, we systematically evaluated the model on 23 different values: 1, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100, 150, and 200 trees.

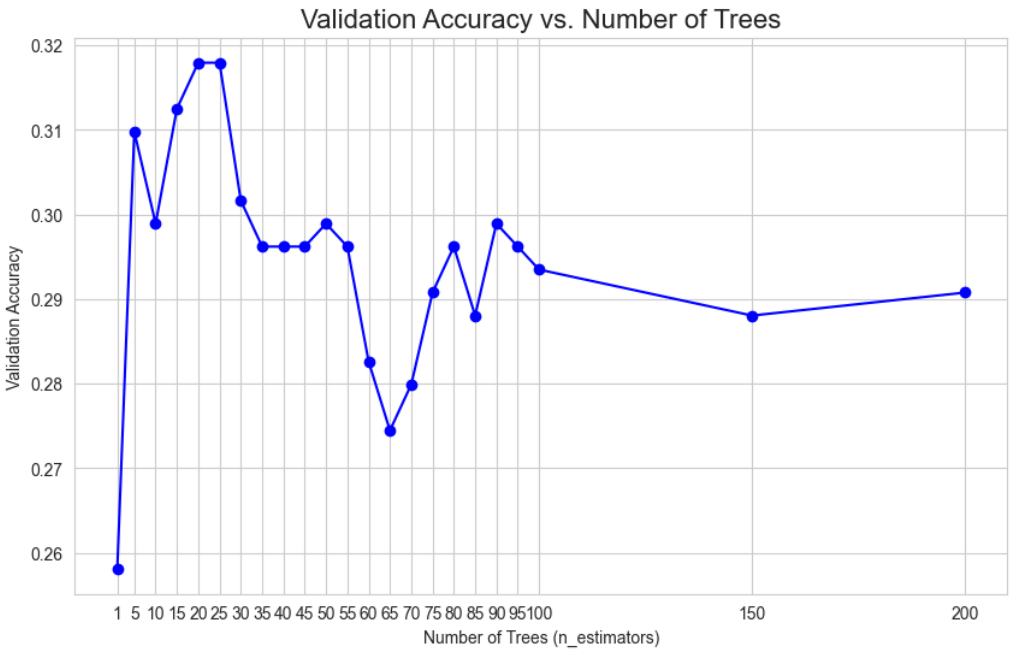

Figure 4: Validation accuracy versus number of trees, showing optimal performance at 20-25 trees

Figure 4 reveals fascinating dynamics in the validation curve. Starting from a single tree (essentially a decision tree with 25.82% accuracy), performance rapidly improves as we add trees, reaching a peak of 31.79% at 20 trees. Interestingly, accuracy plateaus and even slightly degrades beyond 25 trees, fluctuating between 27.5% and 30.2% for larger ensemble sizes. This pattern suggests that after approximately 20 trees, additional trees provide diminishing returns and may even introduce noise through excessive voting. The optimal configuration balances accuracy and computational efficiency at 20 trees, which we use for final training.

4.2 Hyperparameter Sensitivity and Overfitting Considerations

The choice of hyperparameters profoundly influences both Random Forest and SVM classifiers' susceptibility to overfitting or underfitting. Understanding these relationships is crucial for building robust models, especially with limited training data.

For **Random Forest**, the number of trees (`n_estimators`) primarily affects variance rather than bias. With very few trees (e.g., 1-5), the model exhibits high variance—predictions are unstable and heavily dependent on which specific training samples appear in the bootstrap samples. This underfitting manifests as poor performance even on training data. Our validation curve demonstrates this: a single tree achieves only 25.82% accuracy. As we add trees, predictions stabilize through averaging, reducing variance without increasing bias. However, excessive trees (beyond optimal) don't cause traditional overfitting in Random Forest because each tree votes equally, and more trees simply provide more stable estimates of the ensemble's inherent decision boundary. The performance plateauing beyond 20-25 trees suggests we've reached sufficient stability.

The `max_depth` parameter controls individual tree complexity. Shallow trees (low max_depth) create simple decision boundaries that may underfit, failing to capture complex patterns in customer behavior. Deep trees (high max_depth or unlimited) can overfit by memorizing training samples, especially problematic with small datasets like ours. We use sklearn's default (trees grown until pure leaves or minimum samples requirements) with balanced weights, which naturally regularizes through bootstrap aggregation.

`max_features` determines how many features each tree considers at each split. Using fewer features (e.g., sqrt of total features) increases diversity between trees by forcing them to focus on different feature subsets, which enhances ensemble performance and reduces correlation between trees. Using all features at every split causes trees to become similar, reducing ensemble benefits and potentially overfitting shared patterns.

`min_samples_split` and `min_samples_leaf` act as direct regularization. Requiring more samples before splitting or in leaf nodes prevents trees from creating overly specific rules based on small data pockets. With our modest dataset, these parameters are crucial—allowing splits based on just 2-3 samples risks memorizing noise rather than learning generalizable patterns.

For **SVM classifiers**, the regularization parameter `C` controls the overfitting-underfitting trade-off directly. Large C values (e.g., 100, 1000) impose minimal penalty for misclassifications, allowing the model to fit training data very closely. This can lead to overfitting, especially in high-dimensional spaces after one-hot encoding our categorical features. The SVM creates complex, twisted decision boundaries trying to perfectly separate classes, but these boundaries don't generalize to new data. Small C values (e.g., 0.01, 0.1) impose strong regularization, resulting in simpler decision boundaries that may underfit by not capturing genuine patterns.

The `kernel` choice also impacts fitting behavior. Linear kernels create simple hyperplane boundaries that may underfit complex, non-linearly separable data. RBF (Radial Basis Function) kernels can model complex non-linear boundaries but are more prone to overfitting, especially with high `gamma` values. High gamma makes the RBF kernel focus on nearby training points, creating very localized decision regions that don't generalize. Low gamma creates smoother, more global decision boundaries.

`class_weight` indirectly affects overfitting through its interaction with C. Balanced weights increase the effective penalty for minority class errors, which can cause the model to overfit minority classes if combined with large C—the model tries desperately to correctly classify every minority sample, potentially memorizing them. Conversely, with small C, balanced weights help ensure minority classes receive adequate attention without overfitting.

In our implementation, we use default C=1.0, RBF kernel (sklearn's default), and balanced class weights, which represents a middle ground: sufficient flexibility to learn complex patterns while maintaining regularization to prevent overfitting our 1,470-sample training set.

4.3 Random Forest Performance Analysis

The Random Forest with 20 trees achieved a validation accuracy of 31.79%, performing comparably to both OVO (32.88%) and OVA (30.71%). This similarity is noteworthy—Random Forest reaches competitive performance without requiring multiclass decomposition strategies or explicit class balancing beyond tree-level weighting.

Table 3: Random Forest Classification Metrics (n=20 trees)

Class	**Precision**	**Recall**	**F1-Score**	**Support**
A	0.3309	0.3966	0.3608	116
B	0.2500	0.2436	0.2468	78
C	0.2642	0.2188	0.2393	64
D	0.3800	0.3455	0.3619	110
Macro Avg	0.3063	0.3011	0.3022	368
Weighted Avg	0.3165	0.3179	0.3162	368

Table 3 shows that Random Forest achieves the best performance for Segment A among all three methods, with recall of 0.40 (compared to 0.18 for OVA and 0.21 for OVO). This suggests that ensemble averaging helps capture A's characteristics more reliably than individual SVM classifiers. Segment D also performs well with F1-score of 0.36, second only to OVO's 0.41.

However, Random Forest struggles more with Segments B and C compared to OVO. Segment C's recall drops to 0.22 (versus OVO's impressive 0.55), indicating the ensemble has difficulty identifying minority class instances despite tree-level balanced weighting. This illustrates an important point: while Random Forest handles imbalance reasonably well, it doesn't necessarily outperform focused binary classifiers for extreme minority classes. The OVO strategy's pairwise approach still offers advantages for the smallest segment.

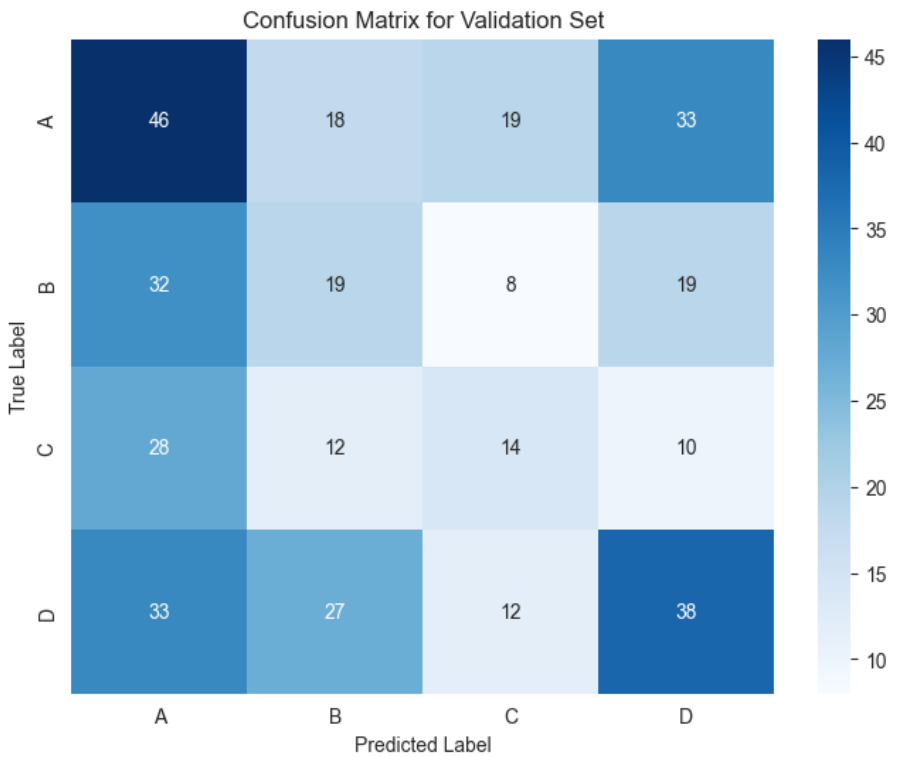

Figure 5: Confusion matrix for Random Forest showing relatively balanced performance across segments

The confusion matrix in Figure 5 displays a more dispersed pattern than either SVM approach, with lower diagonal concentration. Segment A shows 46 correct predictions (versus 24 for OVO and 21 for OVA), confirming Random Forest's strength for this class. However, C shows only 14 correct predictions (versus 35 for OVO), and misclassifications are spread relatively evenly across all classes. This diffuse pattern suggests Random Forest creates less confident decision boundaries—the ensemble's democratic voting sometimes leads to near-ties that could go either way with small perturbations.

4.4 Feature Importance Analysis

One of Random Forest's key advantages is interpretability through feature importance, which measures each feature's contribution to prediction accuracy across all trees. Features that appear in many splits and lead to large purity improvements receive high importance scores.

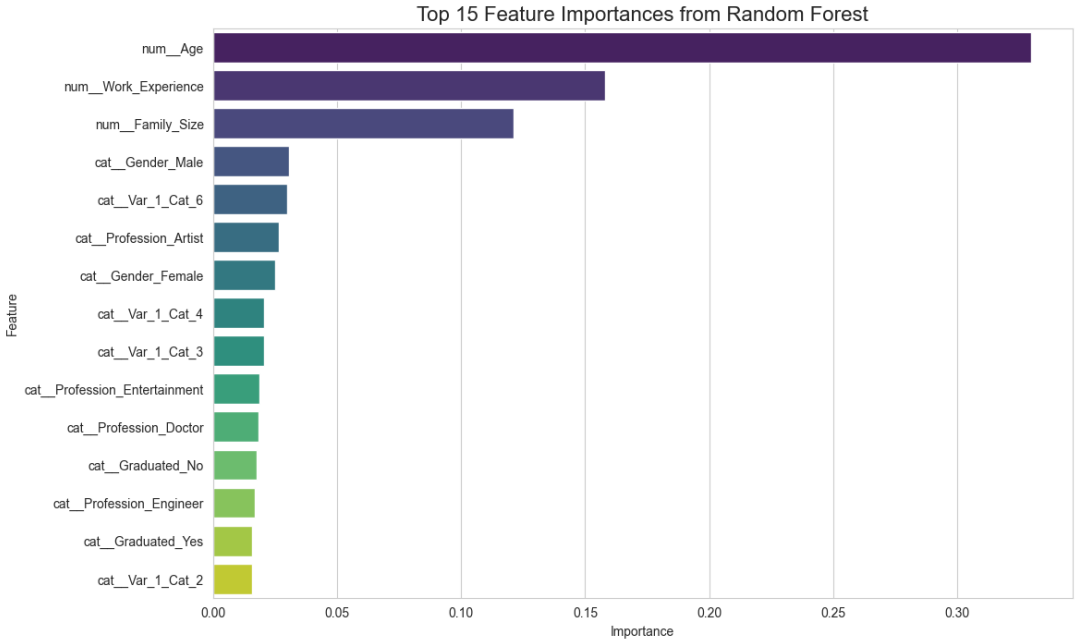

Figure 6: Top 15 features ranked by importance, showing Age as the dominant predictor

Figure 6 reveals that **Age** is by far the most important feature, with importance around 0.32—more than twice the second-ranked feature. This makes intuitive sense for customer segmentation: age correlates strongly with life stage, income level, spending capacity, and behavioral patterns. Different age groups have distinct needs and purchasing behaviors, making age a natural segmenter.

Work Experience ranks second (importance 0.15), which closely relates to age but provides additional information about career progression and earning potential. **Family Size** is the third most important numerical feature (importance 0.12), capturing household structure that influences spending patterns—larger families may prioritize bulk purchases and child-related products, while smaller families might spend more on luxury or experience-based items.

Among categorical features, **Gender (Male)** shows notable importance, suggesting spending behaviors differ significantly between genders. Several one-hot encoded categories from the `Var_1` feature (Cat_6, Cat_4, Cat_3) also appear in the top 15, indicating this variable captures meaningful customer characteristics. **Profession** categories (Artist, Entertainment, Doctor, Engineer) contribute moderately, reflecting how career type influences spending capacity and preferences.

Interestingly, features like `Ever_Married` and `Graduated` show relatively lower importance than one might expect. This could mean these factors are redundant with age and work experience (e.g., age correlates with marriage likelihood), or perhaps customer segments in this dataset aren't strongly differentiated by education level.

This feature importance analysis provides actionable insights: marketing teams should prioritize age-based segmentation strategies, consider work experience when assessing customer lifetime value, and recognize that family size and gender create meaningful sub-segments within age groups. The relatively balanced importance across multiple features also suggests that no single feature perfectly separates the segments—customer behavior is multifaceted, requiring models that consider feature interactions, which Random Forest handles well through its tree structure.

5 Comprehensive Comparison and Conclusions

Having evaluated three distinct approaches to multiclass customer segmentation, we can now synthesize our findings and provide recommendations. Table 4 summarizes the key performance metrics across all methods.

Table 4: Overall Performance Comparison of Classification Methods

Method	**Accuracy**	**Macro F1**	**Weighted F1**	**Best Class**	**Worst Class**
OVA-SVM	0.3071	0.3095	0.3022	D (0.3553)	A (0.2100)
OVO-SVM	0.3288	0.3262	0.3235	D (0.4141)	A (0.2437)
Random Forest	0.3179	0.3022	0.3162	A/D (0.36)	C (0.2393)

One-vs-One (OVO) emerges as the overall best performer, achieving the highest accuracy (32.88%) and most balanced per-class performance. Its superiority stems from the pairwise decomposition that creates simpler, more focused binary problems, particularly benefiting minority classes through reduced imbalance ratios. OVO's 55% recall for Segment C demonstrates exceptional minority class capture—a critical capability for ensuring all customer segments receive appropriate attention in marketing strategies.

Random Forest provides a competitive alternative with valuable interpretability. While its accuracy (31.79%) falls slightly below OVO, it requires no decomposition strategy, trains naturally on multiclass data, and provides feature importance insights that guide business decisions. Random Forest excels at capturing majority class patterns (best recall for Segment A) but struggles more with minority classes compared to OVO's focused approach.

One-vs-All (OVA) lags behind both alternatives, achieving 30.71% accuracy with less consistent per-class performance. The one-versus-rest decomposition creates highly imbalanced binary problems that prove difficult even with balanced class weights. OVA's primary advantage is computational efficiency for problems with many classes, but this benefit doesn't materialize in our four-class scenario.

All three methods perform modestly in absolute terms (31-33% accuracy), which reflects the inherent difficulty of this segmentation task. The overlapping feature distributions, moderate class imbalance, and limited sample size create a challenging learning environment. The small performance differences between methods suggest that no approach has found dramatically better decision boundaries—the data itself may have fundamental ambiguities that limit perfect separation.